

Electricity Sector Data Streaming and Analysis

COMP5339 Project Assignment 2

Zhanhui Lin(550587084), Tangqi Liu(541008068), Jiayin Li(550132552)

I. SYSTEM OVERVIEW

A. Project Execution

The project can be executed locally by installing Python dependencies with `pip install -r requirements.txt`, React dependencies with `npm install` inside frontend, and running `bash start.sh`. The startup script launches Mosquitto on port 1883, FastAPI on port 8000, and the React/Vite dashboard on port 5173. All data is pre-loaded in `data/electricity.db` — no external API calls or streaming delays are required. After startup, the dashboard is available at `http://localhost:5173` and the API documentation at `http://localhost:8000/docs`. The complete project follows an end-to-end data streaming workflow (notebook `COMP5339_Assignment2.ipynb`). OpenElectricity data are retrieved, cleaned, and materialised into CSV files (Tasks 1–2). The publisher replays these records as ordered MQTT messages (Task 3), with an optional continuous loop mode (Task 6). The backend subscriber validates, normalises, stores, and exposes the data through REST endpoints (Task 4). The React dashboard visualises the latest state, historical trends, market data, and the newly connected Assignment 1 records (Task 5).

Figure 1 shows how the submitted system satisfies the assignment workflow from acquisition to visualisation. The pipeline starts with OpenElectricity REST API retrieval for five-minute NEM facility power, CO₂ emissions, and optional regional price/demand. These records are parsed, cleaned, normalised, and cached as CSV files. The streaming layer then publishes combined facility messages through Mosquitto in event-time order. FastAPI subscribes to the MQTT topic, converts each payload into a canonical schema, stores both raw and normalised records in SQLite, and exposes dashboard-ready REST endpoints. The frontend consumes these endpoints through typed API hooks and renders the live map, marker popups, filters, detail panel, bottom analytics, market views, and Assignment 1 historical views.

The Assignment 1 branch is implemented as a separate migration path rather than mixed directly into the streaming loop. Historical state-year energy records and renewable project metadata are migrated from DuckDB into SQLite using `migrate_al.py`; they are then served through `/api/assignment1/*` endpoints. This keeps live MQTT measurements and historical Assignment 1 tables logically separate while allowing the dashboard to combine them in one interface.

The submitted implementation covers all seven assignment tasks: OpenElectricity retrieval and CSV caching (Tasks 1–2), event-time MQTT publishing with a 0.1 s delay (Task 3),

Assignment 1 schema implementation in SQLite (Task 4), map-based subscription and visualisation (Task 5), continuous replay with a 60 s restart delay (Task 6), and this methods/results report (Task 7).

II. DATA ACQUISITION AND MQTT PUBLISHING

A. Data Retrieval

Electricity generation and emissions data were retrieved from the OpenElectricity REST API for the NEM regions of NSW, Queensland, Victoria, South Australia, and Tasmania. The notebook first queried the facilities endpoint with `network_id=NEM` to obtain facility/unit metadata, then requested per-facility power, per-facility CO₂ emissions, and optional regional price/demand at five-minute resolution from 2026-05-01 to 2026-05-08. To stay within the 500-request daily allowance, the retrieval stage used a selected facility-code list spanning gas, coal, distillate, battery, wind, and bioenergy examples. The validation dataset contains 24,192 five-minute facility records across 13 unit identifiers, covering 2026-04-30 14:00 to 2026-05-07 13:55 UTC, which corresponds to the selected May 2026 NEM window.

B. Data Cleaning and Integration

The retrieved JSON responses were transformed into Pandas DataFrames before integration. For power and emissions, the notebook iterated through OpenElectricity result blocks, extracted `unit_code` from each result's columns, and expanded each five-minute data pair into rows containing `unit_code`, `timestamp`, and the measured value. Timestamps were standardised with `pd.to_datetime(..., utc=True)`, and numerical fields such as power, emissions, price, and demand were converted with `pd.to_numeric(..., errors="coerce")`. Duplicate records and rows without usable `unit_code/timestamp` keys were removed. Power and emissions were then combined with an outer merge on `unit_code` and `timestamp`, preserving unit-level rows even when one metric was missing.

Market price and demand were processed separately because they are region-level rather than facility-level metrics. The notebook reshaped OpenElectricity market response blocks into a long table, grouped by `region` and `timestamp`, removed duplicate rows, filtered out WEM, and rounded numeric values to two decimal places. The final facility records were sorted by `timestamp` and `unit_code`, while market records were sorted by `timestamp` and `region`.

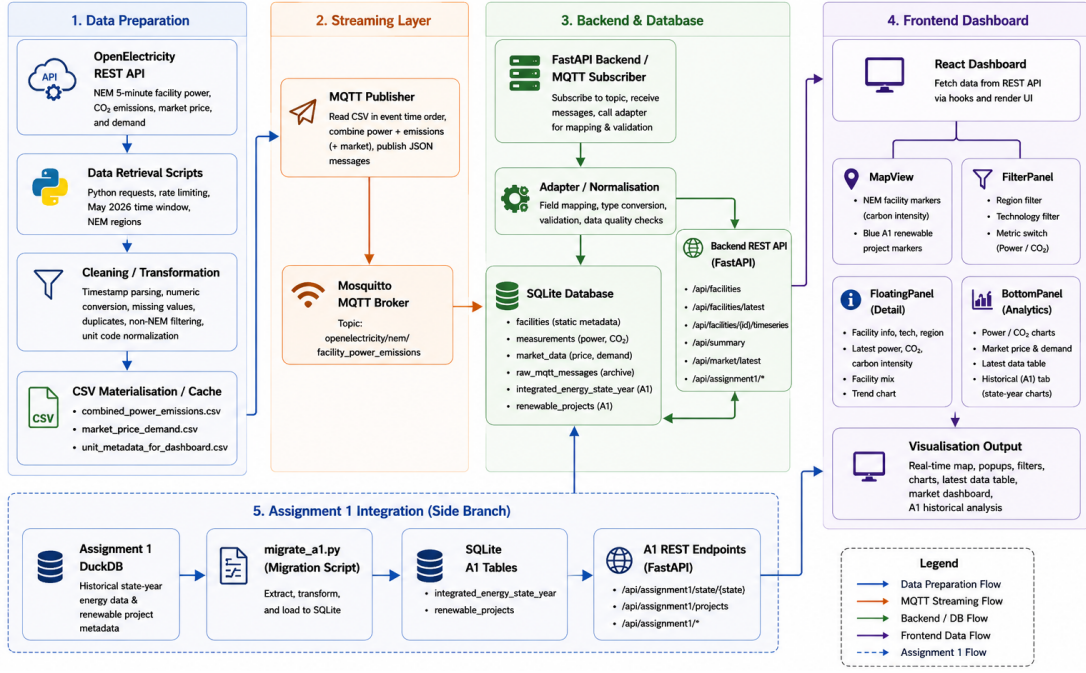


Fig. 1. Overall system architecture of the electricity data streaming and analysis platform.

C. CSV Materialisation

The cleaned outputs were materialised as `combined_power_emissions.csv` and `market_price_demand.csv`. The first file stores unit-level power and emissions records with rounded numeric values; the second stores optional region-level price and demand. In the publishing notebook, unit metadata can also be generated from the previously loaded `facilities_df` and saved as `unit_metadata_for_dashboard.csv`. All files were sorted by timestamp so the publisher can replay records in event-time order. This cache is essential for reproducibility: downstream MQTT and dashboard tests can be repeated without exhausting the OpenElectricity request allowance or changing the input data during a demonstration.

D. MQTT Publishing and Continuous Execution

After Task 2, the cached CSV outputs were replayed as MQTT messages to simulate a facility-level electricity event stream. The publisher uses `combined_power_emissions.csv` as the required input, enriches each row with unit metadata, and optionally joins `market_price_demand.csv` when regional price and demand values are available. A dry-run mode prints representative messages for schema validation before any broker connection is opened, which made it possible to test the publishing format independently from the subscriber.

Before publishing, the cached dataset is converted into a streaming dataframe. Timestamps are parsed using `pandas`, the power and emissions fields are converted to numeric values, duplicate `unit_code`-timestamp rows are removed with the latest row retained, and the records are sorted by timestamp and `unit_code`. The publisher loads unit metadata from `facilities_df` when the notebook is run end-to-end,

otherwise from `unit_metadata_for_dashboard.csv`; if neither exists, it safely falls back to `unit_code`. The `FACILITY_ID_MODE` setting allows the dashboard identifier to be either `unit_code` or `facility_code`, while both identifiers are still retained in the message.

The default MQTT connection uses host `localhost`, port `1883`, and topic `openelectricity/nem/facility_power_emissions`. Each JSON payload includes schema/version metadata, event time, publication time, unit and facility identifiers, facility name, NEM region, fuel technology, status, power in MW, emissions in `tCO2e`, and optional regional price and demand. Messages are published chronologically and the publisher waits `0.1` seconds after each one. For continuous execution, the same publishing pass is wrapped in a replay loop that waits `60` seconds before restarting the next round, simulating an unbounded stream while keeping the API-facing retrieval stage cached and rate-limit safe.

The publisher also includes practical validation controls. `dry_run=True` prints sample JSON messages without connecting to Mosquitto, `max_messages` limits test runs, and `PUBLISH_ONLY_CHANGED_VALUES` can skip unchanged unit values based on power, emissions, price, and demand. These controls helped validate the schema and timing behaviour without accidentally starting an infinite notebook process.

III. MQTT SUBSCRIPTION AND DATA INTEGRATION

A. Subscriber Role in the Pipeline

The MQTT subscriber is the bridge between the simulated streaming layer and the dashboard application. It does not call the OpenElectricity API directly; instead, it listens to the topic used by the publisher and treats every received MQTT payload as one new electricity measurement event. This separation keeps the dashboard independent from the

original data retrieval process and allows the publisher and subscriber to be tested independently.

The subscriber is implemented in the backend service. When the backend starts, it creates an MQTT client, connects to the configured broker, subscribes to the selected topic, and registers a callback function for incoming messages. Each callback execution is intentionally lightweight: the message is decoded, converted into the internal schema, written to the database, and used to refresh the latest in-memory state. The dashboard therefore always reads from the backend's local state instead of connecting to the MQTT broker directly.

B. Incoming MQTT Payload

The MQTT payload is expected to contain the dynamic values required by the dashboard: event timestamp, facility identifier, generated power, and CO₂ emissions. Optional fields may include the facility name, NEM region, fuel technology, market price, and regional demand. Because the publisher's field names may differ from the backend's internal names, the subscriber uses a configurable mapping layer rather than hard-coding a single external schema.

```
{
  "timestamp": "2026-05-10T12:00:00+10:00",
  "facility_id": "BW01",
  "facility_name": "Bayswater Power Station",
  "network_region": "NSW1",
  "fuel_tech": "coal_black",
  "power_mw": 1320.5,
  "emissions_tco2e": 1185.2,
  "price_aud_mwh": 86.4,
  "demand_mw": 7350.0
}
```

Listing 1. Representative MQTT message before normalisation.

The exact external key names may be adjusted after integration with the final publisher. For example, the same information may arrive as `duid`, `unit_code`, `generation_mw`, `co2`, or similar variants. The subscriber resolves these variations through the adapter layer described below.

C. Payload Normalisation

After a message is received, the subscriber first decodes the JSON payload and converts it into a canonical internal record. Table I shows the fields used by the backend after normalisation. This canonical schema is important because the database, API endpoints, and frontend components all depend on stable field names even if the MQTT publisher changes its output format.

The normalisation step also performs basic type conversion. Numeric values are converted to floating-point values, timestamps are kept in ISO format for consistent sorting, and missing optional fields are stored as null values. Invalid messages are not allowed to break the dashboard: they are logged as raw MQTT messages for debugging and skipped from the latest-state cache if mandatory fields such as timestamp, facility identifier, or power output are unavailable.

TABLE I
CANONICAL INTERNAL SCHEMA USED BY THE SUBSCRIBER AND DASHBOARD.

Field	Type	Purpose
timestamp	ISO 8601 string	Event time of the measurement
facility_id	string	Link to facility metadata
power_mw	float	Latest generation output
emissions_tco2e	float	Latest CO ₂ emissions
network_region	string	NEM region filter and grouping
fuel_tech	string	Fuel-technology filter and grouping
price_aud_mwh	float	Optional regional market price
demand_mw	float	Optional regional demand

TABLE II
MAIN BACKEND DATA OBJECTS.

Object	Role
facilities	Static metadata from Assignment 1, including facility name, coordinates, region, and fuel technology.
measurements	Time-series records received from MQTT, including timestamp, facility identifier, power, and emissions.
market_data	Optional regional market price and demand values, keyed by region and timestamp.
raw_mqtt_messages	Raw payload archive used for debugging publisher/subscriber integration.
latest_state	In-memory dictionary containing the newest measurement for each facility.

D. Storage and Latest-State Cache

The backend stores two types of data. First, every valid MQTT measurement is inserted into a time-series table so that the dashboard can display historical trends for a selected facility. Second, the backend maintains a latest-state cache keyed by `facility_id`. This cache stores only the most recent message for each facility and is used by the map view, which only needs the latest power and emissions values to render markers.

The latest-state cache avoids scanning the measurement table every time the frontend refreshes the map. Since the number of displayed facilities is limited, a dictionary lookup by facility identifier is sufficient and avoids the operational complexity of a separate cache service.

E. Integration with Assignment 1 Data

The MQTT stream mainly provides dynamic operational data, while the map dashboard requires static context such as latitude, longitude, facility name, NEM region, and fuel technology. These attributes come from the Assignment 1 facility metadata. The integration key is the facility identifier. When a new MQTT record is processed, its `facility_id`

TABLE III
DASHBOARD-FACING API RESPONSIBILITIES.

API responsibility	Dashboard use
Facility metadata	Supplies coordinates, station names, regions, and fuel technologies for map markers.
Latest measurements	Supplies current power and emissions values for all visible facilities.
Facility time series	Supplies historical trend data after a marker is selected.
Grouped summaries	Supplies regional and fuel-technology totals for bar charts.
Market data	Supplies optional price and demand panels when available.
Assignment 1 history	Supplies 2020–2023 state-year emissions, generation, and renewable project metadata.

is matched against the facility metadata table. The dynamic values from MQTT are then joined with the static metadata before being returned to the frontend.

This integration step is essential for visualisation. Without the Assignment 1 metadata, a MQTT message could indicate that facility BW01 generated a certain amount of power, but the dashboard would not know where to place the marker, how to label the station, or how to group it by region and fuel technology. Conversely, the static metadata alone cannot show real-time changes. The dashboard is therefore built from the combination of static facility records and the latest subscribed measurements.

The project now also implements the broader Assignment 1 schema in the dashboard database. The DuckDB output from Assignment 1 is migrated into SQLite tables for `integrated_energy_state_year` and `renewable_projects`. The first table contains 32 state-year records for 2020–2023 across eight Australian states and territories. The second contains 131 renewable project records, of which 28 include usable latitude and longitude coordinates after geocoding. These historical objects are exposed through three additional API endpoints for summary statistics, state-year time aggregation, and renewable project markers.

IV. DASHBOARD IMPLEMENTATION

A. Backend API for Dashboard State

The frontend does not subscribe to MQTT directly. Instead, it requests processed dashboard state from the backend API. This design keeps MQTT connection management, schema normalisation, and database writes inside the backend, while the frontend receives clean JSON responses designed for visualisation. The most important API responses are the facility metadata list, the latest measurement list, facility-level time series, grouped summaries by region or fuel technology, and optional market price/demand records.

This API boundary also makes the system easier to debug. If the dashboard display is incorrect, the backend API response can be inspected separately from the MQTT broker and the React components.

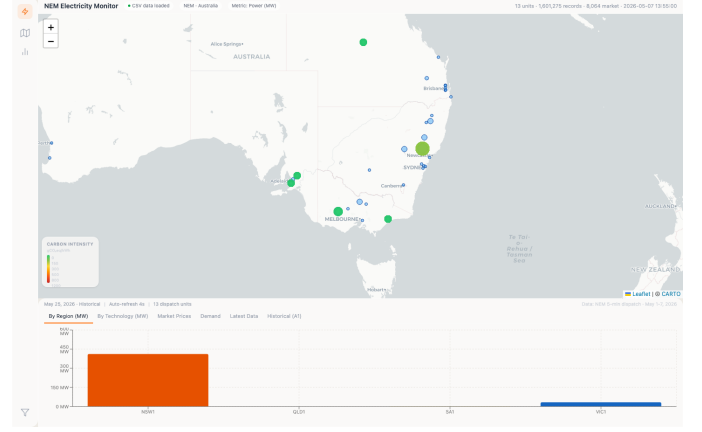


Fig. 2. Main dashboard in power mode, showing carbon-intensity-coloured NEM facility markers together with blue Assignment 1 renewable project markers and the regional power summary.

B. Map-Based Facility Visualisation

The main dashboard view is a map of NEM facilities. Each facility is rendered as a marker using its latitude and longitude from the metadata table. The marker is enriched with the latest MQTT measurement after the frontend merges the static facility list with the latest measurement response. Facilities with no recent measurement can still be shown as inactive markers, while facilities with subscribed measurements display their latest power and emissions values.

The dashboard can switch between power and emissions/carbon-intensity views without changing the underlying data pipeline. It also overlays the 28 geocoded Assignment 1 renewable projects as semi-transparent blue markers, separating historical project metadata from the carbon-intensity-coloured NEM facility markers. Project popups expose name, state, fuel source, status, capacity, project date, and match quality. Figure 2 shows the main power-mode state, with additional states in Appendix A.

C. Interactive Marker Details

When the user clicks a marker, the dashboard opens a detail panel combining station name, identifier, region, fuel technology, latest power, latest emissions, and recent time-series trend requested by `facility_id`. The marker gives the latest subscribed state, while the panel shows recent operational history.

D. Historical Assignment 1 View

The bottom analytics panel now includes a *Historical (A1)* tab. Three frontend hooks retrieve Assignment 1 summary data, state-year records, and geocoded projects. The tab shows state cumulative emissions, emissions intensity, and summary cards for 32 state-year rows, 131 renewable projects, 28 geocoded projects, and a 21.4% geocoding match rate.

E. Filtering and Aggregated Views

The dashboard includes filters for NEM region and fuel technology. The same filtered dataset drives the map and summary charts, allowing users to compare where generation

and emissions are concentrated without inconsistent panel states.

F. Real-Time Update Mechanism

The dashboard uses periodic API refreshes to approximate real-time behaviour. The backend updates local state as MQTT messages arrive, and each frontend refresh updates marker values, detail panels, summary charts, and top-level statistics from the processed API state.

V. DASHBOARD OBSERVATIONS

The integrated dashboard shows two complementary temporal views. The live NEM stream is facility-centred and emphasises current dispatch behaviour, while the Assignment 1 tab provides a longer historical context at state-year granularity. In the captured dashboard state, NSW dominates the current regional power summary among the subscribed dispatch units, with VIC contributing a smaller share. This reflects the limited replay subset rather than the full NEM fleet, but it is useful for validating that the streaming aggregation, filters, and map markers are being updated consistently.

The historical Assignment 1 view shows that QLD, NSW, and VIC account for the largest cumulative emissions over 2020–2023. Emissions intensity is highest in VIC and QLD, indicating that the historical generation mix in those states remains more carbon-intensive than states with lower fossil generation shares. The 131 renewable project records add a planning and infrastructure dimension to the operational dashboard. Only 28 projects are displayed on the map because the dashboard intentionally filters to records with valid coordinates; the resulting 21.4% geocoding match rate is therefore made explicit in the summary card rather than hidden from the user.

VI. CHALLENGES AND SOLUTIONS

A. Changing Message Formats

A practical challenge was that the publisher and subscriber components were developed by different team members. The final MQTT field names were therefore not guaranteed at the start of dashboard implementation. To avoid coupling the dashboard to one exact JSON structure, the backend uses a normalisation layer between the raw MQTT payload and the internal schema. This makes the system more robust: when the publisher changes a field name, only the mapping configuration or adapter logic needs to be updated, while the database, API, and frontend can remain unchanged.

B. Combining Streaming Data with Static Metadata

Another challenge was that the MQTT stream contains dynamic measurements but does not, by itself, provide all information required for a map visualisation. The dashboard needs geographic coordinates and categorical attributes that come from the Assignment 1 metadata. This was solved by treating `facility_id` as the integration key. The backend stores subscribed measurements separately from static facility metadata and joins them when preparing dashboard responses.

C. Migrating and Displaying Assignment 1 Records

The newly connected Assignment 1 data had a different analytical shape from the live MQTT data. State-year emissions and renewable project records are historical and partially geocoded, while MQTT records are time-ordered facility measurements. To avoid rewriting the existing streaming workflow, the Assignment 1 tables were added incrementally to the same SQLite database and exposed through separate REST endpoints. The frontend then added three TypeScript interfaces and three API hooks so that Historical A1 visualisations could be implemented without changing the existing live-data hooks. Renewable projects without coordinates are retained in the summary statistics but excluded from map rendering.

D. Dashboard Consistency During Updates

Because MQTT messages arrive continuously, different dashboard panels must refer to the same latest state. The backend maintains a latest-state cache and exposes processed API responses, so the frontend updates the map, statistics, filters, and charts from one consistent source rather than interpreting raw MQTT messages in multiple components.

VII. FUTURE IMPROVEMENTS

Future work should replace API polling with WebSocket push updates, add a time slider for stream replay, and provide stronger anomaly indicators for unusually high emissions intensity. The Assignment 1 layer would also benefit from improved geocoding so that more than 28 of the 131 renewable projects can be placed on the map. Finally, Docker Compose packaging would simplify repeatable deployment and testing.

VIII. CONTRIBUTION STATEMENT

Zhanhui Lin: Implemented Tasks 4–5, including Assignment 1 SQLite schema migration, MQTT subscriber integration, JSON adapter, REST API endpoints, React map dashboard, filters, marker popups, historical A1 views, and final report integration. **Tangqi Liu:** Implemented Tasks 1–2, including OpenElectricity API retrieval, facility/unit metadata extraction, power/emissions and market-data cleaning, integration, deduplication, and CSV materialisation. **Jiayin Li:** Implemented Tasks 3 and 6, including MQTT topic/schema design, event-time message preparation, dry-run validation, 0.1-second publishing delay, and continuous replay with a 60-second restart delay.

IX. ACKNOWLEDGEMENT OF AI USE

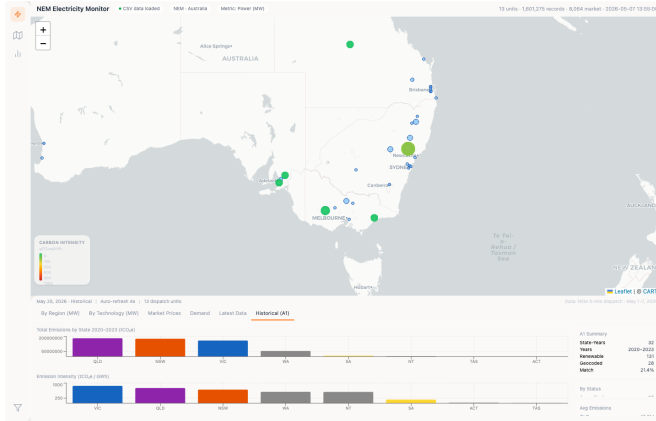
Tool name and version: ChatGPT (GPT-5.5)

Publisher: OpenAI

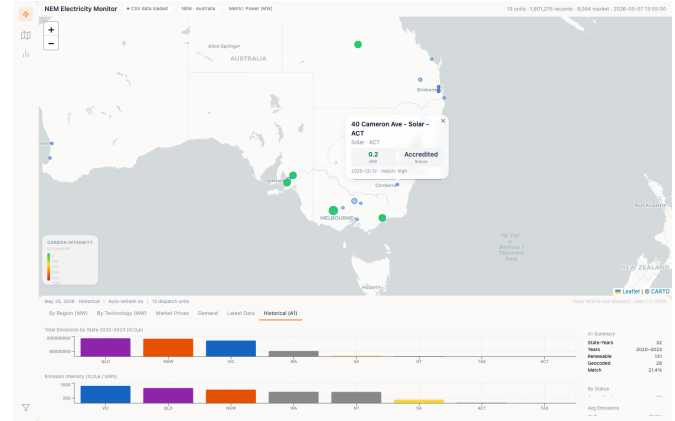
URL: <https://chatgpt.com/>

How it was used: The tool was used to refine and polish the wording of the report for clarity, grammar, and expression. It was also used to generate the system architecture diagram based on the authors' own project design and implementation details.

APPENDIX A DASHBOARD SCREENSHOTS

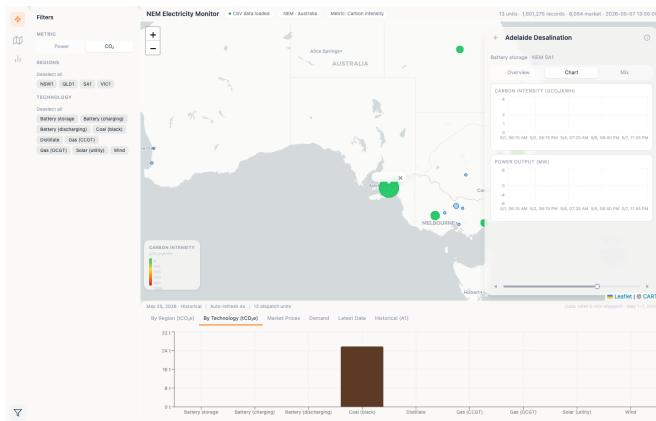


(a) Historical Assignment 1 tab showing cumulative emissions, emissions intensity, and summary cards for 32 state-year rows and 131 renewable projects.

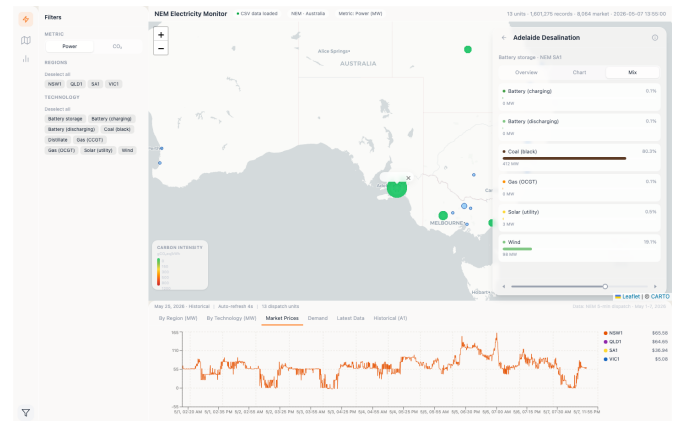


(b) Assignment 1 renewable project popup opened from a blue map marker, showing project metadata, capacity, status, date, and match quality.

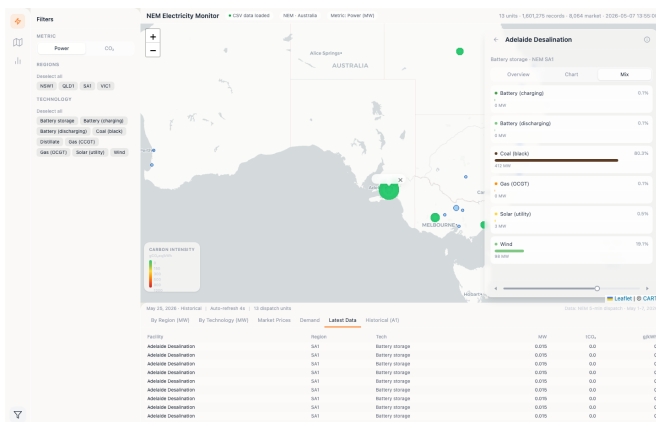
Fig. 3. New Assignment 1 dashboard views added after connecting the historical database tables to the React frontend.



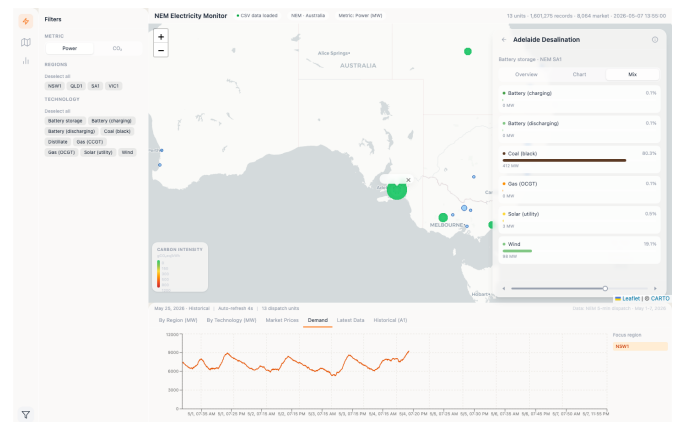
(a) CO₂ mode with facility-level trend charts and emissions by technology.



(b) Market-price view with selected-facility fuel mix panel.



(c) Latest subscribed facility records displayed as a table.



(d) Regional demand time-series panel.

Fig. 4. Additional dashboard screens showing metric switching, facility charts, market data, latest data records, and regional demand trends.